Análise Comparativa da Efetividade de Testes Baseados em Propriedades em APIs REST Java/Spring Boot e Node.js/Express por meio de *Mutation Testing*

Miguel Vieira[1]

[1]Bacharelado em Engenharia de Software
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil
miguel.souza.1447219@sga.pucminas.br

Abstract

Property-Based Testing (PBT) checks general properties over automatically generated inputs, but evidence on its effectiveness comes from single ecosystems, and mutation tools generate different mutants for the same program. This work compared the effectiveness, measured by mutation score, of equivalent PBT suites in two equivalent implementations of an event-ticketing web API, one in Java/Spring Boot (jqwik, PIT) and one in Node.js/Express (fast-check, StrykerJS). A controlled factorial experiment crossed the ecosystem, the strength of the properties (eight mostly bound-checking properties versus a suite that adds nine exact-oracle properties) and the number of tries per property (10, 100 and 1000), with 30 runs per configuration (360 runs), each tool's default mutation operators and a harmonized operator set; differences were tested with Mann-Whitney and Vargha-Delaney effect sizes. Oracle strength dominated: with 100 tries, the mutation score rose from 50.0% to 84.1% in Java and from 52.1% to 95.0% in Node.js. More tries helped only the strong suite. With default operators, Node.js scored higher in every configuration, because PIT mutates return values that state-based properties do not observe; with harmonized operators and 1000 tries, both reached 96.9%. The ecosystem therefore matters less than how properties are written and which operators are counted.

Keywords: *Test oracle. Test suite quality. Controlled experiment. Mutation tools.*

Resumo

Testes Baseados em Propriedades (PBT, do inglês Property-Based Testing*) verificam propriedades gerais sobre entradas geradas automaticamente, mas a evidência sobre sua efetividade vem de um único ecossistema, e ferramentas de mutação diferentes geram mutantes diferentes para o mesmo programa. Este trabalho comparou a efetividade, medida pelo* mutation score*, de suítes PBT equivalentes em duas implementações equivalentes de uma interface de programação web de venda de ingressos, uma em Java/Spring Boot (jqwik, PIT) e outra em Node.js/Express (fast-check, StrykerJS). Um experimento controlado fatorial cruzou o ecossistema, a força das propriedades (oito propriedades que verificam sobretudo limites contra uma suíte que acrescenta nove com oráculo exato) e o número de tentativas por propriedade (10, 100 e 1000), com 30 execuções por configuração (360 execuções), os operadores de mutação padrão de cada ferramenta e um conjunto harmonizado; as diferenças foram testadas com Mann-Whitney e tamanho de efeito de Vargha-Delaney. A força do oráculo foi o fator dominante: com 100 tentativas, o* mutation score *subiu de 50,0% para 84,1% no Java e de 52,1% para 95,0% no Node.js. Mais tentativas só ajudaram a suíte forte. Com os operadores padrão, o Node.js ficou acima em todas as configurações, porque o PIT muta valores de retorno que propriedades baseadas em estado não observam; com operadores*

harmonizados e 1000 tentativas, os dois chegaram a 96,9%. O ecossistema pesa menos do que a forma de escrever as propriedades e os operadores considerados.

Palavras-chave: *Oráculo de teste. Qualidade de suítes de teste. Experimento controlado. Ferramentas de mutação.*

Bacharelado em Engenharia de Software – PUC Minas
Trabalho de Conclusão de Curso (TCC)
Orientadores de conteúdo (TCC I): Danilo Maia, João Pedro Batisteli e Leonardo Vilela
Orientador acadêmico (TCC I): Cleiton Tavares
Orientador do TCC II: Danilo Quadros Maia
Belo Horizonte, 30 de novembro de 2026.

1 Introdução

Testes automatizados são a principal forma de verificar regras de negócio em APIs *backend*, e sua qualidade costuma ser medida pela cobertura de código, que mostra o que foi executado, mas não o que foi verificado [Jain et al., 2023]. Os Testes Baseados em Propriedades (PBT, do inglês *Property-Based Testing*) descrevem uma propriedade geral do sistema e deixam que a ferramenta gere centenas de entradas para tentar refutá-la [Claessen and Hughes, 2000]. Para medir se uma suíte de fato detecta defeitos, o *Mutation Testing* introduz pequenas alterações artificiais no código (mutantes) e verifica quantas delas fazem algum teste falhar; essa proporção é o *mutation score* [Jia and Harman, 2011; Papadakis et al., 2019].

A evidência sobre a efetividade do PBT vem quase sempre de um único ecossistema, e ferramentas de mutação diferentes produzem conjuntos de mutantes diferentes para o mesmo programa [Kintis et al., 2018; Monteiro et al., 2022]. O problema tratado neste trabalho é, portanto: **não se sabe se as mesmas propriedades, aplicadas a implementações equivalentes de uma API REST em Java/Spring Boot e em Node.js/Express e avaliadas pelas ferramentas de mutação de cada ecossistema, têm a mesma efetividade, nem quanto essa efetividade depende do oráculo das propriedades e do número de entradas geradas.**

Responder a essa questão é relevante para quem escolhe um ecossistema e suas ferramentas de teste e para quem compara resultados de mutação entre projetos. Ravi and Coblenz [2025] mostraram, em Python, que um teste de propriedade encontra em média cerca de 50 vezes mais mutações que um teste unitário, mas com grande variação entre tipos de propriedade; Goldstein et al. [2024] relataram que desenvolvedores experientes têm dificuldade em avaliar se suas suítes PBT são efetivas; e Kintis et al. [2018] mostraram que nenhuma ferramenta de mutação para Java subsume as demais. Sem controlar a ferramenta, a forma das propriedades e o número de tentativas, uma diferença de *mutation score* entre linguagens pode refletir qualquer um desses fatores.

O **objetivo geral** é **comparar, por meio do *mutation score*, a efetividade de suítes PBT equivalentes em duas implementações equivalentes de uma API REST em Java/Spring Boot e Node.js/Express, sob diferentes forças de oráculo, números de tentativas e conjuntos de operadores de mutação.** Os objetivos específicos, ligados às questões de pesquisa (QP) da Seção 4.5, são:

OE1 caracterizar as propriedades derivadas das regras de negócio e sua equivalência entre jqwik e fast-check (QP1);

OE2 comparar o *mutation score* entre ecossistemas, suítes e números de tentativas, com operadores padrão e harmonizados (QP2–QP4);

OE3 analisar a relação entre cobertura e *mutation score* (QP5);

OE4 classificar os mutantes não detectados segundo a causa da não detecção (QP6);

OE5 comparar o custo de execução e o tamanho da configuração das ferramentas (QP7).

Como resultados, espera-se obter: (i) evidência sobre o efeito do ecossistema, da força do oráculo e do número de tentativas sobre o *mutation score*, com teste estatístico; (ii) um conjunto de operadores harmonizado que torne comparáveis os mutantes do PIT e do StrykerJS; (iii) a explicação, mutante a mutante, do que as suítes deixam de detectar; e (iv) um pacote de replicação com a especificação, as duas APIs, as suítes, os scripts e os dados brutos.

O restante do artigo está organizado assim: a Seção 2 apresenta os conceitos; a Seção 3, os trabalhos relacionados; a Seção 4, o método; a Seção 5, os resultados e sua discussão; a Seção 6, as ameaças à validade; a Seção 7, as conclusões; e a Seção 8, o pacote de replicação. Os apêndices trazem tabelas e figuras complementares.

2 Fundamentação Teórica

2.1 Testes Baseados em Propriedades

Em PBT, o desenvolvedor escreve uma **propriedade**: uma afirmação que deve valer para todas as entradas de um domínio, como "o total de uma reserva nunca é negativo". Uma **invariante** é um caso particular, uma condição que toda operação deve preservar. A ferramenta usa **geradores** para produzir entradas aleatórias dentro do domínio e executa a propriedade para cada uma; quando encontra um contraexemplo, aplica ***shrinking***, reduzindo-o ao menor caso que ainda falha [Claessen and Hughes, 2000].

As bibliotecas atuais também injetam **casos de borda** (zero, mínimo, máximo, listas vazias) entre as entradas aleatórias. O jqwik, para a JVM, roda sobre a plataforma JUnit 5 [jqwik team, 2026]; o fast-check, para JavaScript e TypeScript, roda dentro de executores como o Jest [Dubien, 2026]. Ambos geram valores aleatórios com semente nova a cada execução, por padrão.

A qualidade de uma suíte PBT depende de duas coisas. A primeira é o **oráculo**, isto é, o que cada propriedade verifica. Uma propriedade que só checa limites ("$0 \leq$ desconto $\leq$ subtotal") aceita qualquer cálculo que respeite esses limites; uma que compara o sistema com um **modelo de referência** verifica valores exatos. A segunda é a **amostra de entradas**: o número de tentativas por propriedade e a forma como o gerador distribui os valores. Um defeito que só aparece em um valor exato, como o último dia de validade de um cupom, só é encontrado se o gerador produzir esse valor. O experimento deste trabalho varia os dois aspectos.

2.2 *Mutation Testing*

O *Mutation Testing* aplica **operadores de mutação** ao programa, cada um gerando uma versão com uma pequena alteração: trocar < por <=, + por −, remover uma chamada, devolver `null`. A suíte é executada contra cada mutante [Jia and Harman, 2011]. O mutante é **morto** quando algum teste falha, **sobrevivente** quando todos passam e **sem cobertura** quando nenhum teste executa o trecho alterado. Mutantes que estouram o tempo limite contam como detectados. O *mutation score* é:

$$MS = \frac{K + T}{K + T + S + N} \times 100\% \tag{1}$$

em que K, T, S e N são os mutantes mortos, com *timeout*, sobreviventes e sem cobertura. Mutantes que não compilam ou quebram a execução ficam de fora, como fazem PIT e StrykerJS.

Um **mutante equivalente** altera o código sem alterar o comportamento observável e, por isso, nunca pode ser morto. Identificá-los exige análise manual [Papadakis et al., 2019]. Além disso, o conjunto de operadores varia entre ferramentas: o PIT muta *bytecode* Java [Coles et al., 2016] e seu conjunto padrão é mais restrito que o recomendado na literatura [Laurent et al., 2017]; o StrykerJS muta a árvore sintática de JavaScript e inclui operadores sem equivalente no PIT, como a troca de literais de texto [Stryker Mutator, 2026]. Comparar ecossistemas exige, portanto, controlar também o conjunto de operadores, restringindo a comparação às transformações que as duas ferramentas aplicam da mesma forma.

3 Trabalhos Relacionados

Foram selecionados seis estudos empíricos de Engenharia de Software que avaliam PBT, a relação entre cobertura e *Mutation Testing* ou ferramentas de mutação; quatro deles foram publicados entre 2023 e 2025. Para cada um, apresentam-se objetivo e método, os resultados relevantes e a relação com este trabalho. A Tabela 1 resume o posicionamento.

Ravi and Coblenz [2025] avaliaram a efetividade de PBT em Python. Analisaram 426 programas que usam a biblioteca Hypothesis, definiram 12 categorias de propriedade e aplicaram *Mutation Testing* às suítes de 40 projetos. Cada teste de propriedade encontrou, em média, cerca de 50 vezes mais mutações que um teste unitário; testes que verificam exceções, pertinência e tipos foram os mais efetivos; e 76% das mutações encontradas apareceram nas primeiras 20 entradas geradas. É o trabalho mais próximo deste. Aqui, a mesma pergunta é feita em dois ecossistemas, com propriedades controladas, e o efeito do número de entradas é medido diretamente (10, 100 e 1000 tentativas).

Goldstein et al. [2024] investigaram como o PBT é usado na prática por meio de 30 entrevistas semiestruturadas na Jane Street, empresa que usa PBT em OCaml, com análise temática. Os participantes concentraram o uso em poucos padrões de alto retorno, como comparação com um modelo de referência e propriedades de ida e volta, e relataram dificuldade em escrever geradores e em avaliar a efetividade das suítes. Este trabalho responde a essa dificuldade de forma quantitativa: mede, por mutação, quanto propriedades

com modelo de referência detectam a mais que propriedades de limite.

Corgozinho et al. [2023] caracterizaram como desenvolvedores implementam propriedades, classificando 86 testes de propriedade de 30 repositórios Python. Propriedades de ida e volta (37,2%) e de oráculo (33,7%) somaram 70,9% dos testes. As categorias catalogadas por esse estudo orientaram a escolha das propriedades da suíte fraca deste trabalho, cuja efetividade é agora medida.

Jain et al. [2023] propuseram a "lacuna de oráculo", a diferença entre cobertura e *mutation score*, e a estudaram em 45 projetos Java, em arquivos com asserções removidas e em código de criptomoedas. Sobre o código coberto, cobertura e *mutation score* tiveram correlação fraca (0,21), e remover asserções ampliou a lacuna. A QP5 deste trabalho mede essa lacuna em PBT e mostra como ela se fecha quando o oráculo das propriedades fica mais forte.

Kintis et al. [2018] compararam quatro ferramentas de mutação para Java (PIT, muJava, Major e PITRV) com análise manual dos mutantes e com defeitos reais. A efetividade variou muito entre as ferramentas, e nenhuma subsumiu as demais. **Monteiro et al. [2022]** compararam PIT e Major: o PIT gerou significativamente mais mutantes, com desempenho um pouco melhor, mas o Major superou o PIT quando os mutantes possivelmente equivalentes foram excluídos. Esses resultados motivam o recorte harmonizado de operadores deste trabalho, que estende a comparação de ferramentas ao par PIT/StrykerJS, entre linguagens.

Tabela 1. Posicionamento dos trabalhos relacionados.

Trabalho	Método e linguagem	PBT	Mutação	Compara ecossistemas
Ravi and Coblenz [2025]	Mineração e mutação, Python	sim	sim	não
Goldstein et al. [2024]	Entrevistas, OCaml	sim	não	não
Corgozinho et al. [2023]	Mineração, Python	sim	não	não
Jain et al. [2023]	Cobertura × mutação, Java	não	sim	não
Kintis et al. [2018]; Monteiro et al. [2022]	Comparação de ferramentas, Java	não	sim	não
Este trabalho	Experimento, Java e Node.js	sim	sim	sim

4 Materiais e Métodos

4.1 Tipo de pesquisa

Quanto à natureza, a pesquisa é **aplicada**, pois busca orientar a escolha e o uso de ferramentas de teste em APIs. Quanto à abordagem, é **mista**: predominantemente quantitativa (*mutation score*, cobertura e tempo, com testes de hipótese), com uma etapa qualitativa na classificação dos mutantes não detectados. Quanto aos objetivos, é **explicativa**, porque procura as causas das diferenças de efetividade. Quanto aos procedimentos, é um **experimento controlado**, com manipulação de fatores e repetição; o raciocínio é **hipotético-dedutivo**,

o ambiente é de **laboratório** e a coleta é **transversal**. O experimento foi escolhido porque só ele permite variar um fator de cada vez (ecossistema, força das propriedades, tentativas) mantendo os demais fixos, o que um estudo observacional de projetos existentes não permitiria.

4.2 Ambiente

As execuções ocorreram em uma máquina virtual com 2 vCPUs Intel Xeon de 2,1 GHz, 7 GB de RAM e Ubuntu 24.04, com duas *threads* de mutação em cada ferramenta. A máquina não tinha acesso ao Maven Central; por isso, jqwik e PIT foram compilados a partir do código-fonte das versões publicadas, e o PIT foi acionado pela mesma API que seu *plugin* Maven usa. Um fluxo de integração contínua (GitHub Actions) executa os testes Java com `mvn test`, incluindo a camada Spring, e o PIT pelo *plugin* oficial; para a suíte fraca com 100 tentativas, ele reproduziu os 126 mutantes e o *mutation score* de 50,0% obtidos no experimento.

4.3 Métodos, técnicas e estratégias

Aplicação de referência. Uma API REST de gestão de eventos e reservas de ingressos, com 20 regras de negócio fixadas em uma especificação única (`SPEC.md`) escrita antes do código: cadastro de eventos, lotes e cupons, cálculo de preço com desconto, disponibilidade de ingressos e transições de status da reserva (Apêndice A). A mesma API foi implementada em Java/Spring Boot e em Node.js/Express, com as mesmas 14 rotas e toda a regra em um serviço de domínio. A equivalência foi verificada por 11 cenários de exemplo compartilhados (125 passos), aprovados nas duas implementações.

Suítes de propriedades. A **suíte fraca** reúne as oito propriedades do TCC I (P1–P8), que verificam 10 das 20 regras; quatro delas só conferem limites ("o total nunca é negativo"). A **suíte forte** acrescenta nove propriedades (P9–P17), derivadas regra a regra do `SPEC.md`, para que toda regra tenha uma propriedade com oráculo exato: o resultado esperado é calculado por um modelo de referência escrito a partir da especificação (Apêndice B). As propriedades e os geradores são equivalentes nos dois ecossistemas.

Fatores e repetições. O experimento cruza o ecossistema (Java com jqwik e PIT; Node.js com fast-check e StrykerJS), a suíte (fraca, forte) e o número de tentativas por propriedade (10, 100, 1000), em 12 configurações executadas 30 vezes cada (360 execuções). Em cada uma das 30 rodadas, as 12 configurações foram executadas em ordem aleatória. A mutação atinge só o domínio, as propriedades chamam o domínio diretamente e a semente aleatória muda a cada execução; as demais decisões de projeto estão no Apêndice C.

Operadores de mutação. Como cada ferramenta tem seus próprios operadores, o fator ecossistema mistura linguagem e ferramenta. Para separá-los, o *mutation score* é calculado em dois recortes sobre as mesmas execuções: o **padrão** de cada ferramenta e o **harmonizado**, que mantém só as cinco classes de mutação aplicadas da mesma forma pelo PIT e pelo StrykerJS (aritmética, limite relacional, negação relacional, condição forçada e incremento; Apêndice C).

Hipóteses e análise estatística. A hipótese do estudo é:

- H_0: o *mutation score* das suítes PBT não difere entre Java/Spring Boot e Node.js/ Express;
- H_1: o *mutation score* das suítes PBT difere entre Java/Spring Boot e Node.js/Express.

H_0 foi testada em cada combinação de suíte, tentativas e recorte de operadores com o teste de Mann-Whitney U bilateral [Mann and Whitney, 1947] sobre as 30 execuções de cada lado. O tamanho do efeito foi medido pela estatística $\hat{A}_{12}$ de Vargha-Delaney [Vargha and Delaney, 2000], a probabilidade de uma execução Java superar uma Node.js ($0{,}5$ indica nenhuma diferença), como recomendam Arcuri and Briand [2011]. Os efeitos da suíte e do número de tentativas foram avaliados com os mesmos testes e com Kruskal-Wallis [Kruskal and Wallis, 1952]. Todos os p-valores foram corrigidos pelo método de Holm [Holm, 1979], a $\alpha = 0{,}05$. Como as execuções reavaliam os mesmos mutantes, a diferença entre ecossistemas também foi testada por qui-quadrado com o mutante distinto como unidade.

Classificação dos mutantes não detectados. Cada mutante não detectado em ao menos uma das 90 execuções de uma combinação de ecossistema e suíte recebeu uma de quatro classes: **entrada não gerada** (a propriedade detecta o defeito, mas só se o gerador produzir a entrada exata que o revela), **regra sem propriedade**, **oráculo fraco** (a regra é verificada, mas a propriedade não observa a diferença) ou **equivalente**. As classes partem das taxas de detecção e de um mapa entre cada linha do domínio e a regra que ela implementa; os mutantes que nenhuma suíte detectou foram lidos um a um. A planilha com a classe e a justificativa de cada mutante foi revisada pelo autor, que confirmou todas as classes.

4.4 Fluxograma da metodologia

A Figura 1 mostra as etapas do estudo, descritas a seguir.

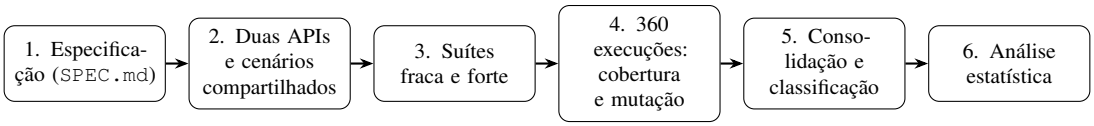


Figura 1. Etapas da metodologia.

1. **Especificação:** as 20 regras, as propriedades e as faixas dos geradores foram fixadas no `SPEC.md`, fonte única das duas implementações.
2. **Implementações:** a API foi implementada nos dois ecossistemas, e os 11 cenários compartilhados confirmaram a equivalência.
3. **Suítes:** as propriedades P1–P17 foram escritas em jqwik e fast-check e validadas contra as duas implementações; uma rodada piloto revelou duas lacunas da suíte forte (entradas nulas e nomes de tipo), corrigidas antes do experimento.
4. **Execuções:** cada execução rodou a suíte com medição de cobertura e, em seguida, a análise de mutação completa, gravando os relatórios brutos.
5. **Consolidação e classificação:** um *script* consolidou os relatórios em uma linha por execução e uma por mutante, e os mutantes não detectados foram classificados.

6. **Análise:** as questões de pesquisa foram respondidas com estatística descritiva, testes de hipótese e figuras.

4.5 Objetivos, questões e métricas (GQM)

O estudo segue o modelo *Goal-Question-Metric* [Basili et al., 1994]: *analisar* suítes PBT *com o propósito de* compará-las *com respeito à* efetividade medida por *mutation score* *do ponto de vista* do desenvolvedor *no contexto de* duas implementações equivalentes de uma API REST. A Tabela 2 liga cada objetivo específico às questões e métricas; a Tabela 3 detalha as métricas.

Tabela 2. GQM: objetivos específicos, questões de pesquisa e métricas.

OE	Questão de pesquisa	Métricas
OE1	QP1: Quais regras puderam ser expressas como propriedades equivalentes, e com que oráculo?	M1, M2, M3
OE2	QP2: Qual o *mutation score* de cada suíte, e quanto a força do oráculo o altera?	M4, M5
OE2	QP3: O número de tentativas por propriedade altera o *mutation score*?	M5, M6
OE2	QP4: O *mutation score* difere entre os ecossistemas (H_0), com operadores padrão e harmonizados?	M5, M7
OE3	QP5: Os mutantes cobertos pela suíte são também detectados?	M8
OE4	QP6: Onde se concentram os mutantes não detectados, e por quê?	M9, M10
OE5	QP7: Qual ecossistema exige mais tempo de execução e mais configuração?	M11, M12

4.6 Ferramentas

A Tabela 4 lista as ferramentas e versões. O código, a especificação, os *scripts* e os dados brutos estão no pacote de replicação (Seção 8).

5 Resultados e Discussão

5.1 Sumarização dos resultados

Foram realizadas 360 execuções completas, com 82 260 avaliações de mutantes (excluídos os erros de execução). Por execução, o PIT gerou 126 mutantes com seu conjunto padrão e o StrykerJS, 260; no recorte harmonizado, os dois conjuntos ficaram quase do mesmo tamanho (131 e 128), com a mesma distribuição por classe. A Tabela 5 resume cada configuração, e a Figura 2 mostra o *mutation score* em função do número de tentativas.

A configuração da versão anterior deste trabalho (suíte fraca, 100 tentativas) foi reproduzida: 50,0% no Java e 52,1% no Node.js. As figuras complementares (por classe de mutação, por módulo, por propriedade e de tempo) estão no Apêndice D.

Tabela 3. Métricas: definição e fonte.

Id	Métrica	Definição	Fonte
M1	Regras cobertas	Regras verificadas por ao menos uma propriedade da suíte / 20	`SPEC.md`
M2	Propriedades equivalentes	Propriedades com mesmo enunciado e mesmos geradores nos dois ecossistemas	código
M3	Tamanho das suítes	Linhas não vazias e não comentário de propriedades e geradores	código
M4	Contagens de mutantes	Gerados, mortos, *timeout*, sobreviventes, sem cobertura e erros	PIT, StrykerJS
M5	*Mutation score*	$MS = (K + T)/(K + T + S + N)$ (Equação 1), nos recortes padrão e harmonizado	idem
M6	Mutantes instáveis	Mutantes detectados em parte, mas não em todas, as execuções de uma configuração	`mutants.csv`
M7	Teste de H_0	Mann-Whitney U, p-valor corrigido por Holm, $\hat{A}_{12}$ e qui-quadrado por mutante	`runs.csv`
M8	Cobertura	Linhas e ramos cobertos e mutantes cobertos, por módulo	JaCoCo, Istanbul
M9	Mortes por propriedade	Proporção dos mutantes detectados que cada propriedade mata, e que só ela mata	`mutants.csv`
M10	Classe do não detectado	Entrada não gerada, regra sem propriedade, oráculo fraco ou equivalente	classificação
M11	Tempo	Tempo da análise de mutação, total e por mutante avaliado	`runs.csv`
M12	Tamanho da configuração	Arquivos de configuração e artefatos declarados para PBT, mutação e cobertura	repositório

Tabela 4. Ferramentas e versões.

Papel	Java	Node.js
Plataforma e camada REST	Java 17, Spring Boot 3.3.5	Node.js 22.22, Express 5.3
Testes e PBT	JUnit Platform 1.9, jqwik 1.7.4	Jest 30.5, fast-check 4.10.2
Mutação	PIT 1.18.2 + pitest-junit5-plugin 1.2.3	StrykerJS 10.0 + jest-runner
Cobertura	JaCoCo 0.8.12	Istanbul (via Jest)

Tabela 5. *Mutation score* (mediana e intervalo interquartil das 30 execuções) com operadores padrão e harmonizados, cobertura de linhas e ramos e mutantes instáveis (M6), por configuração.

			Mutation score padrão (%)		Harmon.	Cobertura (%)	
Ecossistema	Suíte	Tent.	Mediana	IQR	(%)	linhas / ramos	Instáveis
Java	fraca	10	47,6	46,8–48,4	58,0	89,7 / 72,1	23
		100	50,0	50,0–50,0	60,3	91,1 / 73,3	0
		1000	50,0	50,0–50,0	60,3	91,1 / 73,3	0
	forte	10	70,6	69,0–72,2	76,3	97,8 / 93,0	56
		100	84,1	83,3–85,5	91,6	98,7 / 100,0	20
		1000	88,1	87,3–88,9	96,9	98,7 / 100,0	6
Node.js	fraca	10	50,2	49,8–50,6	51,6	90,7 / 81,8	14
		100	52,1	52,1–52,4	54,7	91,3 / 81,8	4
		1000	52,5	52,5–52,5	54,7	91,3 / 81,8	1
	forte	10	83,8	82,7–84,8	79,7	99,4 / 98,5	74
		100	95,0	94,6–95,4	94,5	100,0 / 100,0	18
		1000	96,1	96,1–96,5	96,9	100,0 / 100,0	2
Java	11 exemplos		92,1	—	94,7	—	—
Node.js	11 exemplos		94,6	—	94,5	—	—

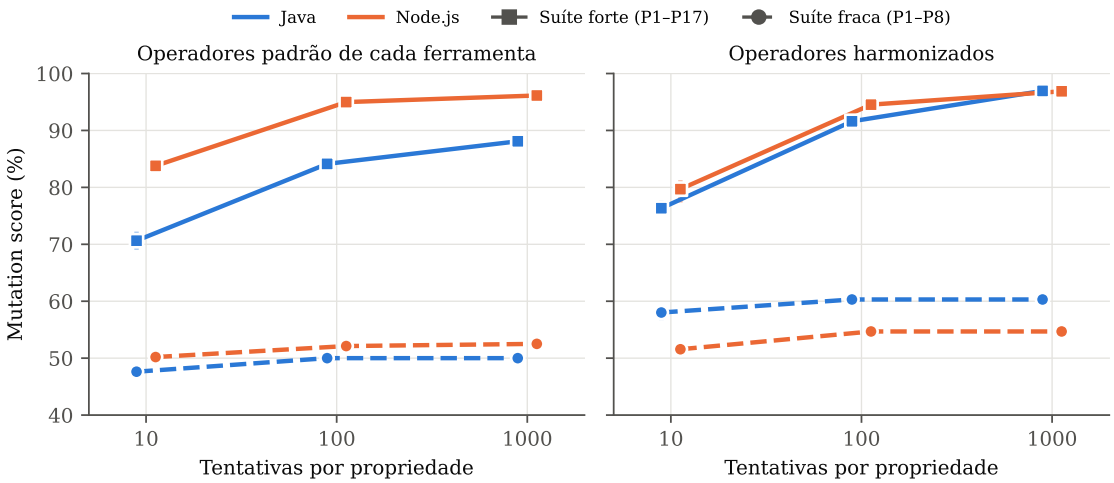


Figura 2. *Mutation score* por número de tentativas, suíte e ecossistema: mediana (marcador) e intervalo interquartil (barra vertical) das 30 execuções. Linhas cheias: suíte forte; tracejadas: suíte fraca.

5.2 Discussão dos resultados

QP1 – Quais regras puderam ser expressas como propriedades equivalentes, e com que oráculo? Todas. As 17 propriedades foram implementadas nos dois ecossistemas com o mesmo enunciado, as mesmas faixas de geração e o mesmo número de tentativas (M2 = 17/17). A suíte fraca verifica 10 das 20 regras (M1 = 50%) e nenhuma de suas propriedades confere o valor do preço; a suíte forte verifica as 20 (M1 = 100%). O tamanho foi equivalente: 336 linhas no Java e 293 no Node.js para a suíte fraca, mais 513 e 446 para as propriedades novas. O jqwik exigiu tipos explícitos para cada dado gerado, enquanto o fast-check compõe geradores em linha; nenhuma propriedade precisou ser adaptada.

QP2 – Qual o *mutation score* de cada suíte, e quanto a força do oráculo o altera? Com 100 tentativas, a suíte fraca detectou 50,0% (Java) e 52,1% (Node.js) dos mutantes, e a suíte forte, 84,1% e 95,0%. A suíte forte foi superior em todas as comparações entre execuções ($\hat{A}_{12} = 1$, $p < 0{,}001$ em todas as combinações), com ganho de 23 a 44 pontos. Os 11 cenários de exemplo detectaram 92,1% e 94,6%, muito acima da suíte fraca, porque conferem valores exatos, inclusive o objeto devolvido por cada operação. Quando as propriedades também conferem valores exatos, elas alcançam os exemplos: no recorte harmonizado, a suíte forte com 1000 tentativas chegou a 96,9% e 96,9%, contra 94,7% e 94,5% dos exemplos.

QP3 – O número de tentativas altera o *mutation score*? Só com propriedades fortes. Na suíte forte, ir de 10 para 100 tentativas elevou o *mutation score* de 70,6% para 84,1% no Java e de 83,8% para 95,0% no Node.js; de 100 para 1000, o ganho foi menor (88,1% e 96,1%). Na suíte fraca, a variação ficou abaixo de 3 pontos, e no Java as 60 execuções com 100 e 1000 tentativas deram o mesmo resultado. Os mutantes que dependem de um valor exato de limite alternam entre detectados e sobreviventes quando há poucas tentativas: na suíte forte, 56 (Java) e 74 (Node.js) mudaram de resultado entre execuções com 10 tentativas, contra 6 e 2 com 1000. Mais entradas só ajudam quando o oráculo reconhece o defeito que elas revelam, o que complementa a observação de Ravi and Coblenz [2025] de que a maioria das mutações aparece nas primeiras entradas.

QP4 – O *mutation score* difere entre os ecossistemas? Depende dos operadores (Tabela 6). Com os operadores padrão, H_0 foi rejeitada em todas as configurações, sempre com o Node.js acima ($\hat{A}_{12} \leq 0{,}01$). A diferença vem da ferramenta: o PIT gera mutantes de valor de retorno (por exemplo, devolver `null` no lugar da reserva atualizada) que as propriedades não observam, porque consultam o estado depois da operação, e o StrykerJS não tem esse operador. No recorte harmonizado, a relação se inverte na suíte fraca (60,3% no Java contra 54,7% no Node.js) e, na suíte forte com 1000 tentativas, H_0 não foi rejeitada (96,9% nos dois, $p = 1{,}000$). Com o mutante distinto como unidade e 100 tentativas, nenhuma diferença harmonizada foi significativa (suíte fraca: $\chi^2 = 0{,}62$, $p = 0{,}430$; forte: $\chi^2 = 0{,}53$, $p = 0{,}468$). A inversão na suíte fraca tem uma causa concreta: as duas ferramentas geram o mesmo número de mutantes de condição forçada por regra, mas, numa validação composta como "nome nulo ou vazio ou capacidade ≤ 0", o PIT gera pares simétricos (metade força o erro, metade desliga a verificação), enquanto o StrykerJS gera uma versão que força o erro e cinco que desligam partes da verificação. A suíte fraca nunca envia dados inválidos e só percebe as versões que forçam o erro. Isso confirma, para o par PIT/StrykerJS, o que Kintis et al. [2018] e Monteiro et al. [2022] observaram entre

ferramentas Java: o conjunto de operadores muda o *mutation score* mais do que a suíte avaliada.

Tabela 6. Teste de H_0 (Mann-Whitney U bilateral, $n = 30$ por grupo, p corrigido por Holm) e $\hat{A}_{12}$, a probabilidade de uma execução Java superar uma Node.js. Os testes dos efeitos de suíte, tentativas e tempo estão no Apêndice E.

		Operadores padrão			Operadores harmonizados		
Suíte	Tent.	p	$\hat{A}_{12}$	H_0	p	$\hat{A}_{12}$	H_0
Fraca	10	$p < 0{,}001$	0,01	✓ **rejeitada**	$p < 0{,}001$	1,00	✓ **rejeitada**
	100	$p < 0{,}001$	0,00	✓ **rejeitada**	$p < 0{,}001$	1,00	✓ **rejeitada**
	1000	$p < 0{,}001$	0,00	✓ **rejeitada**	$p < 0{,}001$	1,00	✓ **rejeitada**
Forte	10	$p < 0{,}001$	0,00	✓ **rejeitada**	$p < 0{,}001$	0,21	✓ **rejeitada**
	100	$p < 0{,}001$	0,00	✓ **rejeitada**	$p < 0{,}001$	0,06	✓ **rejeitada**
	1000	$p < 0{,}001$	0,00	✓ **rejeitada**	$p = 1{,}000$	0,51	✗ não rejeitada

QP5 – Os mutantes cobertos são também detectados? Não, quando as propriedades são fracas. Com 100 tentativas, a suíte fraca cobriu 91,1% das linhas do domínio no Java e 91,3% no Node.js, mas detectou cerca de metade dos mutantes; no módulo de lotes do Java, todos os mutantes foram cobertos e um terço foi detectado (Figura D.2). Com a suíte forte, a cobertura subiu pouco (98,7% e 100,0%), mas o *mutation score* subiu mais de 30 pontos. É a "lacuna de oráculo" descrita por Jain et al. [2023]: a cobertura mostrou onde a suíte não chegava, mas não que metade do código coberto não era verificado.

QP6 – Onde se concentram os mutantes não detectados, e por quê? A Tabela 7 classifica os mutantes não detectados em ao menos uma execução. Na suíte fraca predominam as **regras sem propriedade** (51% no Java e 64% no Node.js), principalmente validações de cadastro, evento inativo e entidade inexistente, seguidas dos **oráculos fracos**. O caso mais claro é o preço: um mutante que faz o subtotal valer sempre zero sobrevive a P1–P4, porque zero é não negativo, um desconto zero está entre zero e o subtotal, a ordem dos itens não muda zero e um cupom inválido não altera zero. Na suíte forte, as regras sem propriedade desaparecem e predominam os mutantes de **entrada não gerada** (82% e 90%), limites exatos que respondem ao número de tentativas; os oráculos fracos restantes são, no Java, valores de retorno não observados. Os **equivalentes** são poucos. O oráculo de preço (P9) foi a propriedade que mais matou nos dois ecossistemas, e as propriedades de limite da suíte fraca (P1, P2, P5) não têm nenhuma morte exclusiva na suíte forte (Figura D.3).

QP7 – Qual ecossistema exige mais tempo e mais configuração? O custo dependeu da suíte (Figura D.4). Com a suíte fraca, o StrykerJS foi mais rápido por mutante (0,11 s contra 0,19 s com 100 tentativas); com a suíte forte, o PIT foi mais rápido no total e por mutante em todos os níveis (74,8 s contra 125,5 s com 1000 tentativas). A configuração tem tamanho parecido: no Java, dois arquivos e quatro artefatos declarados (jqwik, PIT, o *plugin* JUnit 5 do PIT e JaCoCo); no Node.js, quatro arquivos e quatro pacotes. Dois detalhes do Java não são óbvios: sem o *plugin* JUnit 5 o PIT não encontra os testes, e o reaproveitamento de sementes do jqwik precisa ser desligado para que um mutante não influencie o seguinte.

Tabela 7. Mutantes não detectados em ao menos uma das 90 execuções de cada combinação de ecossistema e suíte, por classe (todos os operadores avaliados).

Classe	Principais causas	Suíte fraca		Suíte forte	
		Java	Node.js	Java	Node.js
Regra sem propriedade	Validações; evento inativo; entidade inexistente	54	88	0	0
Oráculo fraco	Preço verificado só por limites; retornos não observados	27	26	11	0
Entrada não gerada	Limites exatos (validade, mínimo, faixas)	23	15	61	75
Equivalente	Identificadores com outro formato; atalho para cupom nulo	2	8	2	8
Total		106	137	74	83

6 Ameaças à Validade

As ameaças foram organizadas pelas quatro validades de Wohlin et al. [2012]. Esta seção apresenta as principais; a lista completa, com a mitigação adotada e a mitigação futura de cada ameaça, está no Apêndice F.

Validade de conclusão. As 30 execuções de uma configuração reavaliam os mesmos mutantes com entradas aleatórias diferentes: medem a variação da geração aleatória, e não a variação entre programas. Por isso, os p-valores devem ser lidos junto com o $\hat{A}_{12}$ e com o teste por mutante distinto, que usa os mutantes como unidade; todos os p-valores foram corrigidos pelo método de Holm. O tempo foi medido em uma máquina virtual compartilhada, e a ordem aleatória das configurações em cada rodada distribui as variações de carga entre elas.

Validade interna. A primeira ameaça é a equivalência entre as implementações, tratada pela especificação única escrita antes do código e pelos 11 cenários compartilhados aprovados nos dois lados. Diferenças próprias de cada linguagem permanecem: o Java devolve objetos que o PIT muta como valores de retorno, enquanto os status do JavaScript são textos que o StrykerJS muta; o recorte harmonizado existe para isolar esses efeitos. A segunda ameaça é a suíte forte ter sido escrita pelo mesmo pesquisador que conhece o código. Ela foi derivada regra a regra do `SPEC.md`, e os dois ajustes feitos após a rodada piloto estão declarados (Seção 4.4). A terceira é a geração de entradas: o jqwik injeta casos de borda que dominam a amostra quando há poucas tentativas, o que faz parte do fator ecossistema mas limita a atribuição das diferenças à linguagem em si.

Validade de construto. O *mutation score* mede a detecção de defeitos artificiais, não de defeitos reais. Como as ferramentas geram conjuntos de mutantes diferentes [Kintis et al., 2018; Monteiro et al., 2022], o fator ecossistema mistura linguagem, biblioteca de PBT e ferramenta de mutação; o recorte harmonizado reduz, mas não elimina, a parte da ferramenta. A classificação dos mutantes não detectados combina critérios objetivos com julgamento manual por leitura do código, confirmado pelo autor, sem cálculo de concordância com um segundo classificador independente; a classe e a justificativa de cada

mutante estão versionadas no repositório.

Validade externa. A aplicação tem cerca de 300 linhas de domínio em cada linguagem, e as propriedades verificam o domínio diretamente, sem passar pela camada HTTP nem por persistência real. Os resultados descrevem o comportamento das ferramentas e dos tipos de propriedade nesse domínio e não devem ser generalizados para APIs grandes sem replicação.

7 Conclusão

Este trabalho comparou a efetividade de suítes PBT equivalentes em duas implementações equivalentes de uma API REST, em Java/Spring Boot e em Node.js/Express, medida pelo *mutation score* em 360 execuções completas de mutação. Quanto à hipótese do estudo, H_0 **foi rejeitada em 11 das 12 comparações**: em todas as seis com os operadores padrão, sempre com o Node.js acima, e em cinco das seis com operadores harmonizados, com o Java acima na suíte fraca e o Node.js acima na suíte forte. H_0 **não foi rejeitada** apenas na suíte forte com 1000 tentativas e operadores harmonizados, em que os dois ecossistemas chegaram a 96,9%. Com o mutante distinto como unidade, nenhuma diferença harmonizada foi significativa. A diferença entre os ecossistemas existe, portanto, mas depende sobretudo da ferramenta de mutação e da força das propriedades, e não da linguagem.

Os objetivos específicos foram atendidos da seguinte forma:

OE1 As 20 regras puderam ser expressas como propriedades equivalentes em jqwik e fast-check, sem adaptação; a suíte fraca verifica 10 delas, e nenhuma de suas propriedades confere o valor do preço.

OE2 A força do oráculo foi o fator dominante: com 100 tentativas, o *mutation score* subiu de 50,0% para 84,1% no Java e de 52,1% para 95,0% no Node.js. Mais tentativas só ajudaram a suíte forte (de 12 a 18 pontos entre 10 e 1000 tentativas, contra menos de 3 na suíte fraca), porque os defeitos de limite só aparecem quando o gerador produz o valor exato e só são reconhecidos quando a propriedade confere esse valor.

OE3 Cobertura alta não significou verificação: a suíte fraca cobriu mais de 91% das linhas do domínio e detectou cerca de metade dos mutantes.

OE4 Na suíte fraca, a maioria dos mutantes não detectados está em regras sem propriedade; na forte, em limites exatos que dependem da entrada gerada. Os equivalentes são poucos (2 no Java e 8 no Node.js).

OE5 O custo dependeu da suíte: com a suíte fraca, o StrykerJS foi mais rápido por mutante; com a forte, o PIT foi mais rápido no total e por mutante. A configuração dos dois ecossistemas tem tamanho parecido.

Para quem testa APIs com PBT, a implicação prática é que as propriedades de limite e de neutralidade, comuns na prática [Corgozinho et al., 2023], protegem pouco os cálculos e as validações: é preciso escrever ao menos uma propriedade com modelo de referência por regra de cálculo e cobrir as regras de validação, e só então aumentar o número de tentativas. Para quem compara ferramentas ou ecossistemas, a implicação é

reportar o *mutation score* sobre um conjunto de operadores harmonizado, e não o total de cada ferramenta.

Trabalhos futuros. (i) Replicar o estudo em outras aplicações, para que a unidade amostral seja o programa e não a execução; (ii) aplicar PBT com estado diretamente sobre a camada HTTP, observando as respostas das operações, o que eliminaria a principal causa de oráculo fraco no Java; (iii) estudar estratégias de geração voltadas a limites derivados da especificação como alternativa a aumentar o número de tentativas; (iv) implementar nas duas ferramentas operadores idênticos, inclusive na direção das condições forçadas; e (v) comparar suítes escritas por desenvolvedores sem acesso à especificação completa com as deste trabalho.

8 Pacote de Replicação

Todo o material do trabalho está no repositório `https://github.com/miguelticio2/tcc-pbt-mutation`:

- `SPEC.md`: as regras R1–R20, as propriedades P1–P17 e as faixas de todos os geradores;
- `java-api/` e `node-api/`: as duas implementações, as suítes de propriedades (fraca e forte) e o executor dos 11 cenários compartilhados (`shared/casos-exemplo.json`);
- `scripts/`: o experimento fatorial (`experimento.sh`), a linha de base com exemplos e a consolidação dos relatórios (`consolidate.py`);
- `data/raw/v2/`: os relatórios brutos das 360 execuções (PIT XML e Stryker JSON, compactados), de cobertura e de tempo; `data/runs.csv` e `data/mutants.csv` com os dados consolidados;
- `analysis/`: a análise estatística, as figuras, as tabelas de resultados e a classificação de cada mutante não detectado, com justificativa;
- `artigo/`: o código-fonte deste artigo, com as macros numéricas geradas pela análise (`numeros.tex`) e a lista das referências dos trabalhos relacionados (`referencias/`);
- `.github/workflows/ci.yml`: a integração contínua, que executa os testes Java (`mvn test`, incluindo a camada Spring) e Node.js e a mutação com o PIT pelo *plugin* Maven oficial e com o StrykerJS, nas duas suítes.

O `README.md` descreve como executar cada etapa com Maven e npm. Os dados da primeira versão do experimento (60 execuções, só a suíte fraca com 100 tentativas) estão preservados no histórico do repositório; a configuração correspondente do experimento atual reproduz os mesmos valores no Java (50,0% em todas as execuções) e a mesma faixa no Node.js (51,7% a 52,5%).

Referências

Andrea Arcuri and Lionel Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE '11)*, pages 1–10. ACM, 2011.

Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. The goal question metric approach. In *Encyclopedia of Software Engineering*. Wiley, 1994.

Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, pages 268–279. ACM, 2000.

Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. PIT: A practical mutation testing tool for Java. In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA 2016)*, pages 449–452. ACM, 2016.

Arthur Lisboa Corgozinho, Marco Tulio Valente, and Henrique Rocha. How developers implement property-based tests. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 380–384. IEEE, 2023.

Nicolas Dubien. fast-check: Property based testing for JavaScript and TypeScript. `https://fast-check.dev`, 2026. Acesso em: 15 out. 2026.

Harrison Goldstein, Joseph W. Cutler, Daniel Dickstein, Benjamin C. Pierce, and Andrew Head. Property-based testing in practice. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*, pages 1–13. ACM, 2024.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Kush Jain, Goutamkumar Tulajappa Kalburgi, Claire Le Goues, and Alex Groce. Mind the gap: The difference between coverage and mutation score can guide testing efforts. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pages 102–113. IEEE, 2023. doi: 10.1109/ISSRE59848.2023.00036.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

jqwik team. jqwik user guide. `https://jqwik.net/docs/current/user-guide.html`, 2026. Acesso em: 15 out. 2026.

Marinos Kintis, Mike Papadakis, Andreas Papadopoulos, Evangelos Valvis, Nicos Malevris, and Yves Le Traon. How effective are mutation testing tools? An empirical analysis of Java mutation testing tools with manual analysis and real faults. *Empirical Software Engineering*, 2018.

William H. Kruskal and W. Allen Wallis. Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association*, 47(260):583–621, 1952. doi: 10.1080/01621459.1952.10483441.

Thomas Laurent, Mike Papadakis, Marinos Kintis, Christopher Henard, Yves Le Traon, and Anthony Ventresque. Assessing and improving the mutation testing practice of PIT. In *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 430–435. IEEE, 2017.

Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1): 50–60, 1947.

Ricardo Monteiro, Vinicius Humberto Serapilha Durelli, Marcelo Eler, and Andre Endo. An empirical analysis of two mutation testing tools for Java. In *Anais do VII Simpósio Brasileiro de Testes de Software Sistemático e Automatizado (SAST)*, pages 49–58, Porto Alegre, 2022. SBC.

Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

Savitha Ravi and Michael Coblenz. An empirical evaluation of property-based testing in Python. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2), 2025. doi: 10.1145/3764068.

Stryker Mutator. StrykerJS documentation. `https://stryker-mutator.io/docs/stryker-js/introduction/`, 2026. Acesso em: 15 out. 2026.

András Vargha and Harold D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.

Apêndice A – Regras de negócio da aplicação de referência

A Tabela A.1 resume as 20 regras do `SPEC.md` por módulo, e a Figura A.1 mostra as transições de status da reserva.

Tabela A.1. Módulos e regras de negócio da aplicação de referência.

Módulo	Regras	Conteúdo
Eventos	R1, R2	Capacidade > 0 e nome não vazio; só eventos `ATIVO` aceitam reservas, itens e confirmações.
Lotes	R3–R5	Preço ≥ 0 e quantidade > 0; soma das quantidades dos lotes $\leq$ capacidade do evento; $0 \leq$ disponível $\leq$ total.
Reservas	R6–R8	Reserva nasce `CRIADA`; itens só em reserva `CRIADA`, com quantidade > 0 e lote do mesmo evento; subtotal $= \sum$ preço $\times$ quantidade.
Cupons	R9–R13	Percentual em [1, 100] ou fixo > 0; desconto $= \lfloor$subtotal $\times$ $\%/100 \rfloor$ ou valor fixo, limitado ao teto; desconto zero se o cupom não existe, expirou ou o subtotal está abaixo do mínimo; $0 \leq$ desconto $\leq$ subtotal.
Disponibilidade	R14–R17	Confirmar exige itens e evento ativo; verifica todos os lotes antes de baixar qualquer um (tudo ou nada); cancelar ou expirar reserva confirmada devolve as quantidades.
Status	R18–R20	Transições da Figura A.1; estados finais não mudam; entidade inexistente gera erro de "não encontrado".

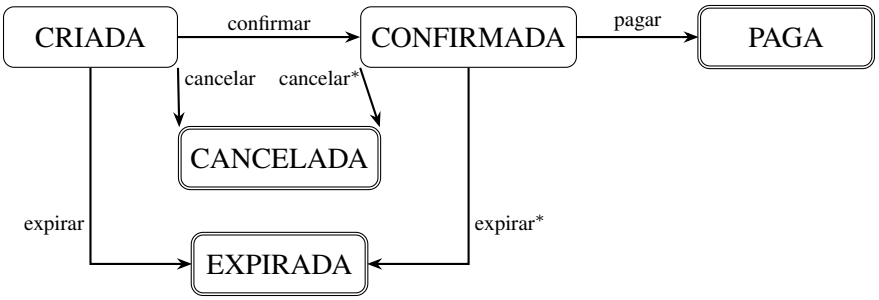


Figura A.1. Transições de status da reserva (R18). Estados com borda dupla são finais (R19). *Devolve a disponibilidade dos lotes (R17).

Apêndice B – Propriedades das suítes

A Tabela B.1 lista as 17 propriedades, as regras que cada uma verifica e o tipo de oráculo. O enunciado completo e as faixas dos geradores estão no `SPEC.md`.

Tabela B.1. Propriedades, regras verificadas e tipo de oráculo. P1–P8 formam a suíte fraca; P1–P17, a suíte forte.

Propriedade	Enunciado	Regras	Oráculo
P1 Total não negativo	Para qualquer reserva e cupom, total ≥ 0	R8, R11–R13	limite
P2 Desconto limitado	$0 \leq$ desconto $\leq$ subtotal e total $=$ subtotal $-$ desconto	R11, R13	limite
P3 Comutatividade	Permutar os itens não altera o subtotal	R8	metamórfico
P4 Cupom inválido neutro	Cupom inexistente, expirado ou abaixo do mínimo não altera o total	R12	neutralidade
P5 Disponibilidade não negativa	Após confirmações, $0 \leq$ disponível $\leq$ total	R5, R15	limite
P6 Redução correta	Confirmação aceita reduz exatamente o pedido; recusada não altera lotes	R15, R16	valor exato
P7 Cancelamento restaura	Confirmar e cancelar (ou expirar) devolve a disponibilidade inicial	R17	ida e volta
P8 Transições válidas	Sequências de operações seguem a Figura A.1 ou são recusadas sem efeito	R18, R19	modelo
P9 Oráculo de preço	Subtotal, desconto e total iguais aos do modelo	R8, R11–R13	modelo
P10 Validação de evento	Evento inválido é recusado; válido nasce `ATIVO` com os mesmos dados	R1, R2	valor exato
P11 Lote e capacidade	Cada cadastro de lote é aceito ou recusado exatamente como no modelo de capacidade	R3–R5	modelo
P12 Validação de cupom	O cadastro aceita exatamente os cupons válidos e preserva os dados	R9	valor exato
P13 Itens da reserva	Adicionar item segue a ordem de verificação e registra o item aceito	R7	valor exato
P14 Evento inativo e reserva vazia	Só evento ativo aceita reservas, itens e confirmações; reserva vazia não confirma	R2, R6, R14	valor exato
P15 Cupom em reserva criada	Aplicar cupom só em reserva `CRIADA`; nos demais status, recusa sem efeito	R10	valor exato
P16 Entidade inexistente	Toda operação sobre id inexistente falha com o código da entidade	R20	valor exato
P17 Modelo de estoque	Estoque, status e erros iguais aos do modelo após cada operação de uma sequência	R15–R19	modelo

Apêndice C – Operadores de mutação e decisões de projeto do experimento

O PIT foi executado com a união dos grupos `DEFAULTS`, `STRONGER` e do operador `NEGATE_CONDITIONALS`; o StrykerJS, com seu conjunto padrão. Sobre os mesmos

resultados, dois recortes são calculados por mutante, o que é válido porque cada mutante é avaliado isoladamente:

- **Padrão**: o conjunto padrão de cada ferramenta, como um desenvolvedor a usaria (`DEFAULTS` no PIT 1.18; todos os operadores no StrykerJS).
- **Harmonizado**: apenas as cinco classes de mutação que existem com a mesma transformação nas duas ferramentas (Tabela C.1). O operador de igualdade do StrykerJS foi dividido em limite ($< \leftrightarrow <=$) e negação ($=== \rightarrow !==$, $< \rightarrow >=$) conforme o operador original e o substituto.

Tabela C.1. Classes de mutação do recorte harmonizado. Ficam fora do recorte os operadores sem equivalente: valores de retorno e remoção de chamadas no PIT; literais, blocos, chamadas de método e operadores lógicos no StrykerJS.

Classe	PIT	StrykerJS
Aritmético	`Math`, `InvertNegs`	`ArithmeticOperator`, `AssignmentOperator`, `UnaryOperator`
Limite relacional	`ConditionalsBoundary`	`EqualityOperator` ($<\leftrightarrow<=$, $>\leftrightarrow>=$)
Negação relacional	`NegateConditionals`	`EqualityOperator` (demais substituições)
Condição forçada	`RemoveConditional` (`EQUAL`/`ORDER`, `IF`/`ELSE`)	`ConditionalExpression`
Incremento	`Increments`	`UpdateOperator`

As demais decisões de projeto do experimento foram:

- **Mutação só no domínio.** PIT e StrykerJS mutam apenas a camada de domínio; controladores e configuração do framework não contêm regra de negócio.
- **Propriedades chamam o domínio diretamente**, não via HTTP, para que o tempo medido não inclua o custo de cada servidor web.
- **Dinheiro em centavos inteiros** nos dois lados, evitando divergências de ponto flutuante que não são do ecossistema.
- **Configurações equivalentes:** geração aleatória e semente nova a cada execução nos dois lados. No jqwik, o reaproveitamento da semente que falhou na execução anterior foi desligado, para que um mutante não influencie o próximo. O número de tentativas é passado às duas bibliotecas por configuração (`jqwik.tries.default` e `numRuns`).
- **Matriz completa de mortes:** o PIT registra todos os testes que matam cada mutante (*full mutation matrix*) e o StrykerJS não para no primeiro teste que falha (`disableBail`), o que permite atribuir cada morte a uma ou mais propriedades.

- **Ordem aleatória:** em cada uma das 30 rodadas, as 12 configurações foram executadas em ordem aleatória (semente = número da rodada), para que variações de carga da máquina não se concentrem em uma configuração.

Apêndice D – Figuras complementares

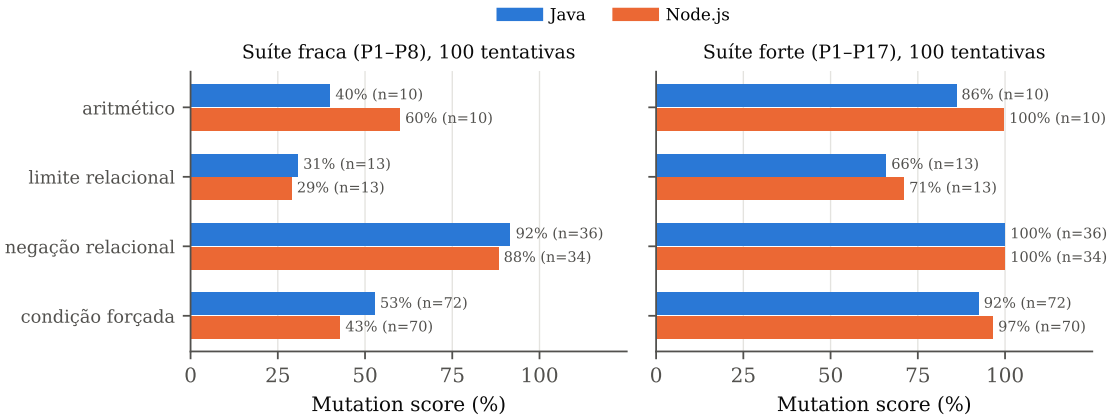


Figura D.1. *Mutation score* por classe de mutação do recorte harmonizado (100 tentativas). Entre parênteses, o número de mutantes da classe por execução. A classe incremento, com um único mutante no Node.js, foi omitida.

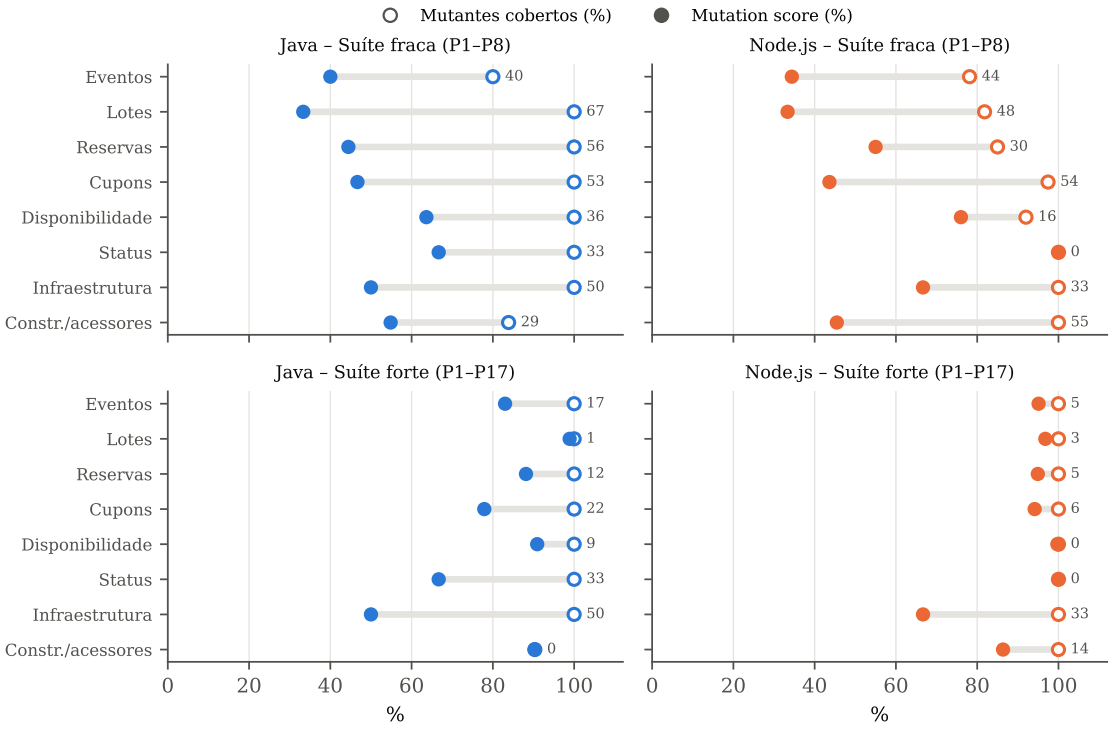


Figura D.2. Mutantes cobertos pela suíte × *mutation score*, por módulo (100 tentativas, operadores padrão). O rótulo à direita é a diferença em pontos percentuais (QP5).

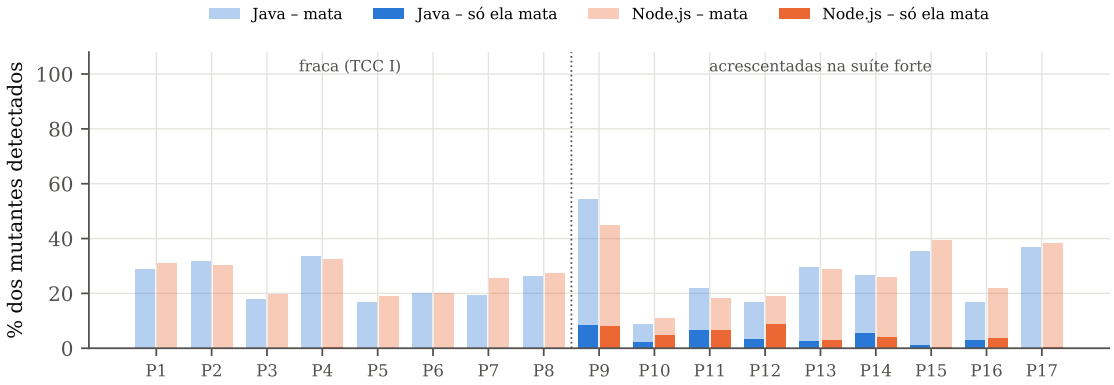


Figura D.3. Suíte forte, 100 tentativas: porcentagem dos mutantes detectados que cada propriedade mata (clara) e que só ela mata (escura), média por execução (QP6).

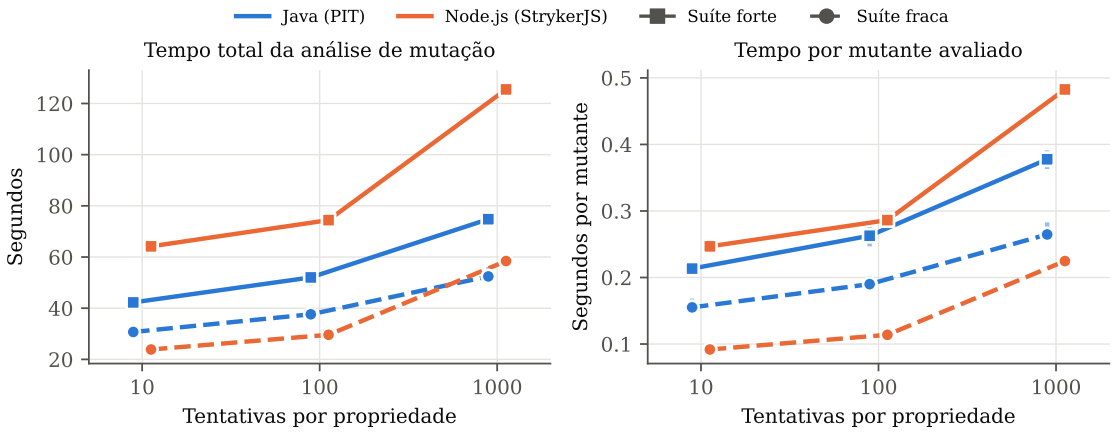


Figura D.4. Tempo da análise de mutação, total e por mutante avaliado: mediana (marcador) e intervalo interquartil (barra) das 30 execuções (QP7).

Apêndice E – Testes estatísticos complementares

Além de H_0, os efeitos da suíte, do número de tentativas e o tempo foram comparados com o mesmo procedimento (Mann-Whitney U bilateral, $n = 30$ por grupo, $\hat{A}_{12}$ e p corrigido por Holm sobre os 38 testes do estudo). Essas comparações descrevem os fatores controlados e não constituem hipóteses adicionais. Na Tabela E.1, $\hat{A}_{12}$ é a probabilidade de uma execução do primeiro grupo superar uma do segundo.

Tabela E.1. Comparações complementares. A última coluna indica se a igualdade foi rejeitada a $\alpha = 0{,}05$.

Comparação	Contexto	Grupos	$\hat{A}_{12}$	p	Igualdade
Suíte	Java, 10 tent.	forte × fraca	1,00	$p < 0{,}001$	✓ **rejeitada**
Suíte	Java, 100 tent.	forte × fraca	1,00	$p < 0{,}001$	✓ **rejeitada**
Suíte	Java, 1000 tent.	forte × fraca	1,00	$p < 0{,}001$	✓ **rejeitada**
Suíte	Node.js, 10 tent.	forte × fraca	1,00	$p < 0{,}001$	✓ **rejeitada**
Suíte	Node.js, 100 tent.	forte × fraca	1,00	$p < 0{,}001$	✓ **rejeitada**
Suíte	Node.js, 1000 tent.	forte × fraca	1,00	$p < 0{,}001$	✓ **rejeitada**
Tentativas	Java, fraca	100 × 10	1,00	$p < 0{,}001$	✓ **rejeitada**
		1000 × 100	0,50	$p = 1{,}000$	✗ não rejeitada
Tentativas	Java, forte	100 × 10	1,00	$p < 0{,}001$	✓ **rejeitada**
		1000 × 100	0,99	$p < 0{,}001$	✓ **rejeitada**
Tentativas	Node.js, fraca	100 × 10	1,00	$p < 0{,}001$	✓ **rejeitada**
		1000 × 100	0,83	$p < 0{,}001$	✓ **rejeitada**
Tentativas	Node.js, forte	100 × 10	1,00	$p < 0{,}001$	✓ **rejeitada**
		1000 × 100	0,99	$p < 0{,}001$	✓ **rejeitada**
Tempo total	fraca, 10 tent.	Java × Node.js	0,99	$p < 0{,}001$	✓ **rejeitada**
Tempo/mutante	fraca, 10 tent.	Java × Node.js	1,00	$p < 0{,}001$	✓ **rejeitada**
Tempo total	fraca, 100 tent.	Java × Node.js	1,00	$p < 0{,}001$	✓ **rejeitada**
Tempo/mutante	fraca, 100 tent.	Java × Node.js	1,00	$p < 0{,}001$	✓ **rejeitada**
Tempo total	fraca, 1000 tent.	Java × Node.js	0,12	$p < 0{,}001$	✓ **rejeitada**
Tempo/mutante	fraca, 1000 tent.	Java × Node.js	1,00	$p < 0{,}001$	✓ **rejeitada**
Tempo total	forte, 10 tent.	Java × Node.js	0,00	$p < 0{,}001$	✓ **rejeitada**
Tempo/mutante	forte, 10 tent.	Java × Node.js	0,00	$p < 0{,}001$	✓ **rejeitada**
Tempo total	forte, 100 tent.	Java × Node.js	0,00	$p < 0{,}001$	✓ **rejeitada**
Tempo/mutante	forte, 100 tent.	Java × Node.js	0,09	$p < 0{,}001$	✓ **rejeitada**
Tempo total	forte, 1000 tent.	Java × Node.js	0,00	$p < 0{,}001$	✓ **rejeitada**
Tempo/mutante	forte, 1000 tent.	Java × Node.js	0,00	$p < 0{,}001$	✓ **rejeitada**

O teste de Kruskal-Wallis entre os três níveis de tentativas rejeitou a igualdade nas quatro combinações de ecossistema e suíte (Java fraca: $p < 0{,}001$; Java forte: $p < 0{,}001$; Node.js fraca: $p < 0{,}001$; Node.js forte: $p < 0{,}001$). Na suíte fraca, porém, as diferenças são menores que 3 pontos percentuais (Tabela 5): com 30 execuções e variância quase nula, diferenças pequenas já são significativas, e por isso o tamanho do efeito e a diferença absoluta devem ser lidos junto com o p-valor.

Apêndice F – Ameaças à validade

Tabela F.1. Ameaças à validade, mitigações adotadas e mitigações futuras.

Validade	Ameaça	Mitigação adotada	Mitigação futura
Conclusão	As 30 execuções reavaliam os mesmos mutantes; variância baixa em algumas configurações	$\hat{A}_{12}$, correção de Holm, qui-quadrado com o mutante distinto como unidade	Mais aplicações, para que a unidade amostral seja o programa
Conclusão	Tempo medido em máquina virtual compartilhada	Ordem aleatória das configurações em cada rodada; mediana; tempo por mutante	Repetir em máquina dedicada
Interna	Implementações não equivalentes	Especificação única, 11 cenários compartilhados (125 passos) aprovados nos dois lados	Testes diferenciais gerados automaticamente entre as duas APIs
Interna	Propriedades mais fortes em um lado	Mesmos enunciados, geradores e tentativas, mapeados 1:1	Revisão das propriedades por um segundo pesquisador
Interna	Suíte forte ajustada aos mutantes	Propriedades derivadas do `SPEC.md`; ajustes do piloto declarados (Seção 4.4)	Suíte escrita por quem não vê os mutantes
Interna	Ferramentas Java compiladas do código-fonte	Mesmas versões publicadas; CI com o *plugin* Maven oficial reproduziu os resultados	—
Construto	*Mutation score* como medida de efetividade	Complementado por cobertura, análise por módulo e classificação	Usar defeitos reais injetados
Construto	Conjuntos de operadores diferentes entre PIT e StrykerJS	Recorte harmonizado com cinco classes de mesma transformação	Implementar operadores idênticos nas duas ferramentas
Construto	Classificação dos não detectados	Critérios objetivos, mapa linha–regra versionado, justificativa por mutante, revisão e confirmação pelo autor	Classificação independente com cálculo de concordância
Externa	Um domínio pequeno, propriedades no nível do domínio	Domínio com regras numéricas, de estado e de disponibilidade	Replicar em APIs de outros domínios, via HTTP